

To write the output of the Sed command to a file, use shell redirection, as follows:

```
$ sed 's/MATCH/REPLACE/' filename > output.txt
```

To save the changes to the input file itself, use the `-i` option:

```
$ sed -i 's/MATCH/REPLACE/' filename
```

In text replacement operations such as the above example, only the first occurrence of the matched text in a line is replaced. If you have multiple occurrences of the search pattern on a single line, and need to replace all occurrences, append the `'g'` (global replacement) switch as follows:

```
$ sed 's/MATCH/REPLACE/g' filename
```

However, sometimes we may not need to replace the first `'N'` occurrences, but replace only the rest of them. There is an inbuilt option to ignore the first `'N'` occurrences and replace from the `'N+1th'` occurrence onwards. This is easiest to explain with examples, which make it very clear:

```
$ echo this thisthisthis | sed 's/this/THIS/2g'
this THISTHISTHIS
$ echo this thisthisthis | sed 's/this/THIS/3g'
this thistHISTHIS
$ echo this thisthisthis | sed 's/this/THIS/4g'
this thistHISTHIS
```

Deleting lines based on how they match

To remove lines that are matched by a regular expression pattern, use the `'d'` suffix. For example, `$ sed '/$/d' filename` removes all blank lines present in the file from the output.

Printing only pattern-matched lines

We have seen different examples like the substitution of text. In the above examples, Sed makes some changes to matched lines, and prints them—or deletes them. What if we want to print only lines that match the pattern, and print only changed text? To print only matched lines, use the `-n` (no output/silent) flag, and attach a `p` suffix after the pattern, to cause printing of lines that match. Let's take a sample file named *paragraph.txt*:

```
1. introduction
some text goes here. 2nd line.
2. next heading
some text goes here. 2nd line. 3rd line.
```

On this file, we run the following command:

```
$ sed -n '/[0-9]\+\.p/' paragraph.txt
```

```
1. introduction
2. next heading
```

Thus, we printed only titles (lines with numbering) from the file.

If we want the whole input text printed, with matched text printed twice, we can use `/p` without the `-n` flag. Try it out: `$ seq 20 | sed '/0/p'`. Lines with 10 and 20 (containing 0) are printed twice.

Matched string reference in replacement text

In Sed, the first block of substitution is the pattern to be matched. Sometimes we need to replace matched text with some additional characters. Here, we can reference the matched text using the `'&'` notation in the replacement block. For example, to add a space between every character in a line, use the following code:

```
$ echo this is a line of text | sed 's/./& /g'
t h i s   i s   a   l i n e   o f   t e x t
```

Each matched character is replaced with itself plus a space. The `/g` option applies this to every character matched in the line.

Sub-string match and back-referencing

We have seen how to reference and use the entire matched pattern in the replacement text. Now, how can we use sub-strings of the matched pattern in the replacement text? For example, we have (matched text): *Lakshman Sarath Mr*. This is last name, first name, and salutation. Let's suppose we want to rewrite this line to read with the salutation first, then the first name, and then the last name (think of several hundred such names in a file, which are in the 'wrong' order). If we can match each word as a sub-string, we can reference these sub-strings individually, and can specify a new order for them in the replacement text.

Sub-strings can be specified by using grouping operators—parentheses—like `'(SUBSTR)'`. They are referenced (in replacement text) by `IN`, where `N` is the `Nth` sub-string found. The grouping operators `'('` and `')'` must be prefixed with the `\` escape character. Let us use back-referencing to solve our problem of re-ordering the parts of a name:

```
$ echo Lakshman Sarath Mr | sed 's/\(\\w\\+\) \(\\w\\+\) \(\\w\\+\)/\3 \2 \1/'
Mr Sarath Lakshman
```

The above (and other Sed scripts) may look complex and hard to understand initially. But once you break it up and analyse it, it is simple to understand. `\w` is the regex used to match character (refer to Table 3 above) and `\+` to specify one or more characters. Each of the words is grouped with `(` and